

HASSAN II UNIVERSITY
FACULTY OF SCIENCES BEN M'SICK
MATHEMATICS AND COMPUTER SCIENCE DEPARTMENT
- CASABLANCA -

MASTER DATA SCIENCE AND BIG DATA (DSBD)
MACHINE LEARNING PROJECT

Cartographie Sémantique et Analyse des Publications de la FSBM par NLP & Web Scraping

Student:
Achraf CHAHID

Supervisor:
Prof. Habib BEN LAHMAR

Contents

1	Data Engineering & Pipeline Architecture	2
1.1	Overview	2
1.2	Robust Data Extraction (Bronze Layer)	3
1.3	OpenAlex Metadata Enrichment	5
1.4	Automated PDF Acquisition via Selenium	5
1.5	Data Cleaning and Structuring (Silver Layer)	5
2	Semantic Search & NLP Integration	7
2.1	Vector Embeddings	7
2.2	Vector Database (ChromaDB)	7
3	System Architecture & Web Application	9
3.1	System Architecture Overview	9
3.2	Backend Infrastructure (FastAPI)	9
3.2.1	Core API Endpoints	9
3.3	Frontend Interface & Application Views (Next.js)	10
3.3.1	Semantic Search & Discovery Engine	10
3.3.2	Faculty Researcher Directory	11
3.3.3	Individual Researcher Profile	12
4	Conclusion & Perspectives	13
4.1	Summary of Achievements	13
4.2	Future Work	13

Chapter 1

Data Engineering & Pipeline Architecture

1.1 Overview

The data engineering phase forms the critical foundation of the semantic search engine. To ensure data integrity, reproducibility, and high performance, the pipeline was designed following the **Medallion Architecture** (Bronze, Silver, Gold). This multi-stage approach guarantees that the system is idempotent, resilient to network interruptions, and respectful of external server rate limits while transforming unstructured web data into structured analytical datasets.

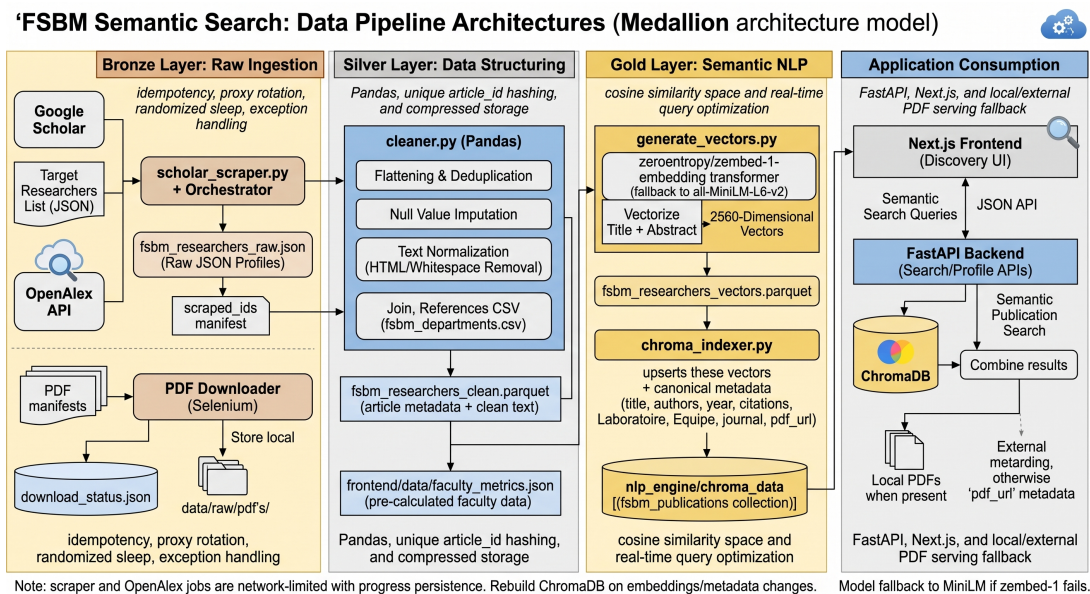


Figure 1.1: Overview of the Medallion Data Pipeline (Bronze to Gold layers)

Detailed Architecture Breakdown

Figure 1.1 provides a comprehensive architectural blueprint of the FSBM Semantic Search engine. The system is structured around the Medallion data design pattern (Bronze, Silver, Gold) followed by a decoupled Application Consumption layer. This pipeline ensures data quality, fault tolerance during network-limited operations, and scalable machine learning inference.

1. Bronze Layer (Raw Ingestion): This initial stage focuses on robust data extraction from external sources. Driven by a curated **Target Researchers List (JSON)**, the `scholar_scraper.py` orchestrator queries Google Scholar and the OpenAlex API. To bypass strict anti-bot network limitations, the layer enforces idempotency via a `scraped_ids`

manifest, proxy rotation, randomized sleep intervals, and strict exception handling. Concurrently, a Selenium-based PDF downloader acquires full-text documents, tracking progress locally in a `download_status.json` manifest. The primary output is the raw, unstructured JSON profiles (`fsbm_researchers_raw.json`).

2. Silver Layer (Data Structuring): The raw, nested data is passed to the `cleaner.py` script, which utilizes the Pandas library to sanitize and structure the dataset for machine learning. Core operations include flattening nested structures, deduplication via unique `article_id` hashing, null value imputation, and text normalization (HTML and whitespace removal). The cleaned data is then joined with departmental references (`fsbm_departments.csv`). The outputs are exported into highly compressed Parquet format (`fsbm_researchers_clean.parquet`) for the NLP engine, alongside pre-calculated faculty metrics (`faculty_metrics.json`) intended directly for the frontend.

3. Gold Layer (Semantic NLP): This layer transforms the structured text into machine-readable mathematical representations. The `generate_vectors.py` module uses the `zeroentropy/zembed-1-embedding` transformer model to convert combined paper titles and abstracts into 2560-dimensional dense vectors. A fallback mechanism to `all-MiniLM-L6-v2` is included in case the primary model fails. These vectors are temporarily stored in a Parquet file before being processed by `chroma_indexer.py`. This script upserts the vectors along with canonical metadata (title, authors, year, citations, laboratory, journal, and PDF URL) into the local ChromaDB vector database, creating a cosine similarity space optimized for real-time querying.

4. Application Consumption Layer: The final tier exposes the Gold layer data to end-users through a modern web stack. A FastAPI backend handles semantic search queries and profile retrieval from ChromaDB, communicating via a JSON API with the Next.js frontend (Discovery UI). Furthermore, the system employs a robust document serving fallback mechanism: when a user requests a paper, the backend serves the locally scraped PDF if it is present in the `data/raw/pdfs/` directory, otherwise, it dynamically redirects to the external metadata `pdf_url`.

1.2 Robust Data Extraction (Bronze Layer)

A major initial hurdle was the inconsistency of the faculty data. Many researchers did not maintain public Google Scholar profiles, and relying on dynamic name searches resulted in high false-positive rates due to name collisions. To solve this, we manually curated a definitive target list, identifying and extracting the exact Google Scholar IDs for verified faculty members. This clean baseline was formatted as a JSON manifest (`target_researchers.json`), ensuring the scraper only targeted validated profiles.

The extraction phase involves querying these profiles using the `scholarly` Python library. Given the strict anti-bot mechanisms (such as CAPTCHAs and HTTP 429 Too Many Requests errors) deployed by Google Scholar, the scraper (`scholar_scraper.py`) and its orchestrator were engineered with high resilience:

- **Idempotency & Crash Recovery:** An atomic JSON-based manifest system (`scraped_ids` set) tracks successful extractions. If the script crashes or is manually halted, it seamlessly resumes exactly where it left off, preventing duplicate work and data corruption.
- **Rate Limiting & Politeness:** Randomized sleep intervals (`random.uniform(5.0,`

10.0)) are injected between profile requests to simulate human pacing and avoid triggering heuristic security triggers.

- **Proxy Rotation:** The integration of `ProxyGenerator` distributes requests across free proxy pools, effectively hiding the host IP address and bypassing IP-based temporary bans.
- **Exception Handling:** Comprehensive `try/except` blocks wrap the network calls, ensuring that a failure on a single publication or profile does not halt the entire pipeline.

```

Terminal Local Local (2) Local (3) Local (4) + v
(.venv) PS C:\Users\chahid\PycharmProjects\Django_Project\fsbm-semantic-search> python data_pipeline/src/orchestrator.py
[*] Fetching and configuring free proxies... (This may take a minute)
[*] Proxy configured successfully! Hiding real IP.
[*] Loaded 95 target profiles.
[*] Found 74 profiles already scraped. Resuming...

=====
[*] Processing Profile 1/95: Habib BEN LAHMAR
=====
[*] Skipping Habib BEN LAHMAR - Already successfully scraped.

=====
[*] Processing Profile 2/95: Nawal Sael
=====
[*] Skipping Nawal Sael - Already successfully scraped.

=====
[*] Processing Profile 3/95: Said Nouh
=====
[*] Skipping Said Nouh - Already successfully scraped.

=====
[*] Processing Profile 4/95: KHALID KANDALI
=====
[*] Skipping KHALID KANDALI - Already successfully scraped.

=====
[*] Processing Profile 5/95: faouzia Benabbou
=====
[*] Skipping faouzia Benabbou - Already successfully scraped.

=====
[*] Processing Profile 6/95: Mohamed Talea
=====
[*] Skipping Mohamed Talea - Already successfully scraped.
  
```

```

Terminal Local Local (2) Local (3) Local (4) + v

=====
[*] Processing Profile 76/95: SAID GHOUH
=====
[*] Skipping SAID GHOUH - Already successfully scraped.

=====
[*] Processing Profile 77/95: SIHAM SLASSI
=====
[*] Skipping SIHAM SLASSI - Already successfully scraped.

=====
[*] Processing Profile 78/95: SOUMIA MORDANE
=====
[*] Fetching Google Scholar profile directly for ID: hINOScAAAAJ
[*] Found profile: Soumia Mordane. Fetching detailed metrics...
[*] Fetching top 15 articles (with rate limiting)...
-> [1/15] Fetching article: An improved coastal upwelling index from sea surface temperature using satellite-based approach-The case of the Canary Current upwelling system
-> [2/15] Fetching article: Offshore wind energy resource in the Kingdom of Morocco: Assessment of the seasonal potential variability based on satellite data
-> [3/15] Fetching article: Impact of the variational assimilation of ground-based GNSS zenith total delay into AROME-Morocco model
-> [4/15] Fetching article: Radar rainfall estimation in Morocco: Quality control and gauge adjustment
-> [5/15] Fetching article: Impact des vents sur l'upwelling au sud du Maroc; apport du modèle ROMS forcé par les données ALADIN et QuikSCAT
-> [6/15] Fetching article: A parabolic equation based on a rational quadratic approximation for surface gravity wave propagation
-> [7/15] Fetching article: Optical and thermo-plasmonic properties of spherical and hollow gold nanoparticles injected in cell organelles
-> [8/15] Fetching article: Numerical analysis of surface plasmon resonance and photothermal properties of silica/gold/graphene and hollow/gold/graphene core-multi-shell nanoparticles
-> [9/15] Fetching article: Wave energy potential analysis in the Casablanca-Mohammedia coastal area (Morocco)
-> [10/15] Fetching article: Optical and thermoplasmonic properties of core (AuAgI-x)-shell (Au) nanostructures
-> [11/15] Fetching article: Computation of nonlinear free-surface flows using the method of fundamental solutions
-> [12/15] Fetching article: Improving heavy rainfall forecasts by assimilating surface precipitation in the convective scale model AROME: A case study of the Mediterranean event of November 4, 201
7
-> [13/15] Fetching article: Wave reflection by rectangular breakwaters for coastal protection
-> [14/15] Fetching article: Numerical modeling of the wave-plate-current interaction by the boundary element method
-> [15/15] Fetching article: New numerical investigation using meshless methods applied to the linear free surface water waves
[*] Saved data for SOUMIA MORDANE to raw/ folder.
[*] Sleeping for 5.54 seconds before next profile...

=====
[*] Processing Profile 79/95: ABDELNUJO ABOURRICHE
=====
[*] Fetching Google Scholar profile directly for ID: iKIUSQIAAAJ
  
```

Figure 1.2: Terminal output demonstrating idempotent scraping, rate limiting, and automated exception handling during data extraction.

1.3 OpenAlex Metadata Enrichment

While Google Scholar provides excellent baseline metrics, it operates as a closed ecosystem. It heavily obfuscates direct PDF links and does not expose structured reference graphs via its API, making deep semantic analysis difficult.

To bypass these limitations, the pipeline integrates the **OpenAlex API**. By cross-referencing our extracted article titles against the OpenAlex database, we successfully enriched the dataset without triggering Google's aggressive rate limits. This solution provided:

- **Direct PDF Routing:** Reliable Open Access (OA) PDF URLs, which are critical for the downstream automated download phase.
- **Citation Graphs:** Exhaustive reference arrays (`referenced_works`) to map relationships between papers.
- **Standardized Identifiers:** Resolution of DOIs and OpenAlex IDs for global academic tracking.

This enrichment process also utilizes an incremental saving mechanism, writing to `openalex_enrichment.json` after every successful HTTP 200 response to safeguard against API timeouts.

1.4 Automated PDF Acquisition via Selenium

A significant challenge in academic data extraction is publisher Content Delivery Networks (CDNs, e.g., Elsevier, ScienceDirect) returning HTTP 403 Forbidden errors to standard programmatic requests (`requests` library).

To solve this, we implemented a headless automated browser using **Selenium** and `webdriver-manager`. By forcing Chrome to download PDFs directly via the `plugins.always_open_pdf_externally` preference, the pipeline naturally manages JavaScript execution and dynamic security tokens. A local state manifest (`download_status.json`) is maintained to ensure the idempotency of the downloads.

1.5 Data Cleaning and Structuring (Silver Layer)

The raw data extracted (Bronze layer) is highly nested and contains noise. The `cleaner.py` script utilizes the **Pandas** library to strictly structure and sanitize the dataset, ensuring it is primed for Machine Learning integration:

- **Flattening & Deduplication:** The nested JSON profiles are flattened into a strict tabular format. Duplicate publications (often caused by co-authorship within the same faculty) are identified and removed using unique `article_id` hashing.
- **Null Value Imputation:** Missing metadata (e.g., missing journals, empty citation arrays) are properly coerced into native Python `None` or empty lists, preventing generic string artifacts from polluting the NLP vectorization phase.

- **Text Normalization:** Publication abstracts undergo basic Natural Language Processing (NLP) pre-treatment: removal of residual HTML tags, conversion to lower-case, and elimination of erratic whitespace (`abstract_clean`).
- **Optimized Storage (Parquet):** The final cleaned dataset is exported as `fsbm_researchers_clean.parquet`. This columnar storage format drastically reduces file size through compression and significantly accelerates I/O operations when loading data into RAM for the GPU-intensive vectorization step.

```
(.venv) C:\Users\chahid\fsbm_semantic_search\fsbm-semantic-search>python data_pipeline/src/cleaner.py
[*] Starting Cleaning Process (Bronze -> Silver)...
[*] Processing 1066 scraped articles...
[*] Merging with reference department data...
[*] Merging with OpenAlex enrichment data (PDF URLs and References)...
[+] Success! Silver layer rebuilt with 993 articles ready for vectorization.
```

Figure 1.3: Terminal output demonstrating the Pandas data structuring process, including deduplication, text normalization, and Parquet export.

Chapter 2

Semantic Search & NLP Integration

2.1 Vector Embeddings

Traditional keyword-based search is insufficient for academic literature, where concepts can be expressed using diverse terminology. To enable semantic search, we transform textual data (abstracts, titles, and extracted PDF text) into dense vector representations. The system utilizes the `zeroentropy/zembed-1-embedding` transformer model hosted via HuggingFace, providing high-dimensional (2560-dimensional) contextual embeddings.

A significant engineering challenge encountered during this phase was the computational bottleneck of vectorizing the scaled dataset. While the initial baseline run successfully encoded 359 abstracts locally, the corpus quickly scaled to over 900 publication records (yielding approximately 850 valid abstracts after cleaning). Generating 2560-dimensional embeddings for a corpus of this size locally caused severe CPU memory constraints and system slowdowns.

To overcome this hardware limitation, we implemented a cloud acceleration strategy. The computationally expensive batch inference was offloaded to an external cloud environment (Google Colab) equipped with high-RAM and GPU resources. This allowed us to successfully encode the entire dataset without Out-of-Memory (OOM) crashes before writing the final vectors back to the local Gold Parquet layer.

2.2 Vector Database (ChromaDB)

The generated embeddings are stored and indexed using **ChromaDB**. ChromaDB was selected for its efficiency in handling high-dimensional vector similarity searches. When a user submits a query, it is embedded using the same NLP model, and ChromaDB performs a K-Nearest Neighbors (KNN) search using cosine similarity to return the most contextually relevant research papers.

An integration difficulty at this stage was the mismatch between ChromaDB's native mathematical output and user-facing metrics. ChromaDB calculates cosine distance (d), where lower values indicate closer semantic matches. However, end-users expect an intuitive confidence score. To resolve this, we implemented a dynamic score normalization layer in the backend API that translates the distance metric into a percentage match ($S = \max(0, 1 - d) \times 100\%$), ensuring the results are both mathematically sound and easily interpretable in the Next.js UI.

```
(.venv) C:\Users\chahid\fsbm_semantic_search\fsbm-semantic-search>python nlp_engine/src/generate_vectors.py
[*] Starting NLP Vectorization Process (Silver -> Gold)...
[*] Loaded 993 articles for vectorization.
[*] Loading transformer model: zeroentropy/zembed-1-embedding...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████████████████████████████████████████████████████████████████████████| 398/398 [00:09<00:00, 40.84it/s]
Default prompt name is set to 'document'. This prompt will be applied to all inference calls, except if a `prompt` or `prompt_name` parameter is provided.
[*] Encoding 993 abstracts... (This may take a minute depending on your CPU/GPU)
Batches: 100%|██████████████████████████████████████████████████████████████████████████████| 125/125 [6:34:27<00:00, 189.34s/it]

[+] Success! All abstracts vectorized.
[+] Vector Dimension: 2560
[+] Gold layer saved to: C:\Users\chahid\fsbm_semantic_search\fsbm-semantic-search\data_pipeline\data\processed\fsbm_researchers_vectors.parquet

(.venv) C:\Users\chahid\fsbm_semantic_search\fsbm-semantic-search>python nlp_engine/src/chroma_indexer.py
[*] Starting ChromaDB Indexing Process...
[*] Loaded 993 vectorized articles.
[*] Loaded 1066 article records from raw lookup.
[*] Connected to ChromaDB Collection: 'fsbm_publications'
[*] Inserting 993 documents into ChromaDB...
100%|██████████████████████████████████████████████████████████████████████████████| 2/2 [00:01<00:00, 1.65it/s]

[+] Success! All vectors indexed in ChromaDB.
[+] Database saved locally at: C:\Users\chahid\fsbm semantic_search\fsbm-semantic-search\nlp_engine\chroma_data
```

Figure 2.1: Terminal output demonstrating the local vectorization baseline and the upsertion of dense embeddings into the ChromaDB vector store.

Figure 2.1 illustrates the execution log from the final corpus run. Following the exhaustive faculty scraping and data cleaning phases, 993 article abstracts were successfully loaded and encoded into 2560-dimensional dense vectors using the `zeroentropy/zembed-1-embedding` model. As shown in the logs, performing local batch inference for a dataset of this scale is highly computationally intensive, requiring approximately 6.5 hours to complete the vectorization. Finally, the indexing script successfully upserted all 993 vectorized documents into the local ChromaDB `fsbm_publications` collection, finalizing the semantic space for the application.

Chapter 3

System Architecture & Web Application

3.1 System Architecture Overview

The semantic discovery platform is designed as a decoupled, micro-service oriented web application. The core system architecture separates compute-intensive vector operations from presentation concerns, facilitating independent scalability, modular maintenance, and low-latency interaction.

- **Data & Vector Engine:** ChromaDB handles vector persistence and approximate nearest neighbor (ANN) retrieval using cosine distance spaces, underpinned by embeddings pre-computed via the `zeroentropy/zembed-1-embedding` transformer model.
- **High-Performance API Backend:** Implemented with **FastAPI**, orchestrating semantic queries, metadata extraction, dynamic similarity score normalization, and PDF document serving.
- **Interactive User Interface:** A reactive, modern Single-Page Application (SPA) built with **Next.js** (React) and styled with **Tailwind CSS**, delivering interactive filtering, real-time query rendering, and responsive academic metrics visualization.

3.2 Backend Infrastructure (FastAPI)

The backend acts as the bridge between the persisted data lakehouse (Parquet and ChromaDB) and the client application. FastAPI was chosen due to its asynchronous ASGI architecture, automatic OpenAPI/Swagger documentation, and native Pydantic data validation.

3.2.1 Core API Endpoints

The backend exposes three principal RESTful routes:

1. **GET /search:** Accepts a natural language query string alongside optional faceted filtering parameters (e.g., publication year, author ID). It dynamically encodes the prompt into dense vector space, queries ChromaDB for the top- k nearest neighbors, and normalizes cosine distance (d) into human-interpretable percentage match confidence ($S = \max(0, (1 - d)) \times 100\%$).

2. **GET /faculty-profiles:** Returns aggregated faculty metadata, research laboratory affiliations, and cumulative bibliometric indicators (*h*-index, *i10*-index, citation counts).
3. **GET /faculty-profiles/{id}:** Retrieves deep-dive publication records, historical output, and OpenAlex enrichment identifiers for an individual academic profile.

3.3 Frontend Interface & Application Views (Next.js)

The frontend is architected using the Next.js App Router, leveraging client-side state management and server-rendered layout performance. The interface is organized around three primary user journeys.

3.3.1 Semantic Search & Discovery Engine

The primary discovery interface provides an intuitive search bar allowing researchers and academic reviewers to query the scientific corpus conceptually rather than relying on exact keyword matching.

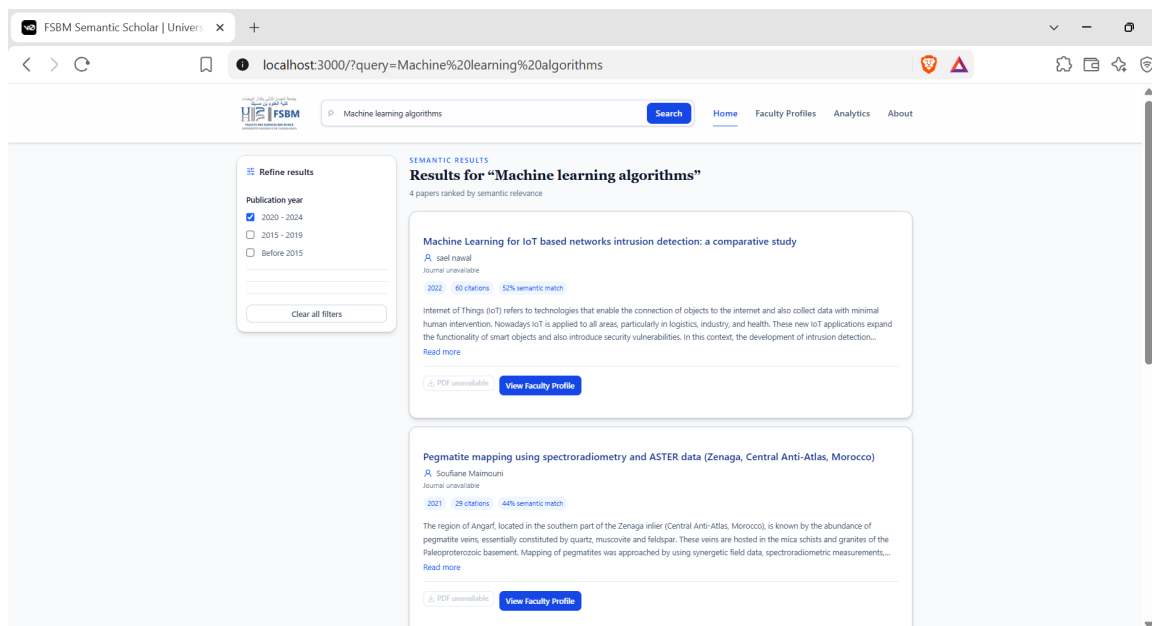


Figure 3.1: Semantic search interface demonstrating multi-faceted filtering by publication year, semantic confidence score badges, and paper metadata cards.

As shown in Figure 3.1, the search results view incorporates:

- **Faceted Navigation:** Dynamic sidebar filters allowing queries to be restricted by publication cohorts (e.g., 2020–2024, 2015–2019, or prior to 2015).
- **Semantic Match Badges:** Dynamic visual indicators showing normalized semantic relevance (e.g., 52% match) alongside total citation counts.
- **Direct Document Access:** Instant status indicators showing Open Access PDF availability and contextual links directly into the respective researcher's profile.

3.3.2 Faculty Researcher Directory

To allow exploratory browsing across departments and laboratories, the application provides a centralized institutional directory.

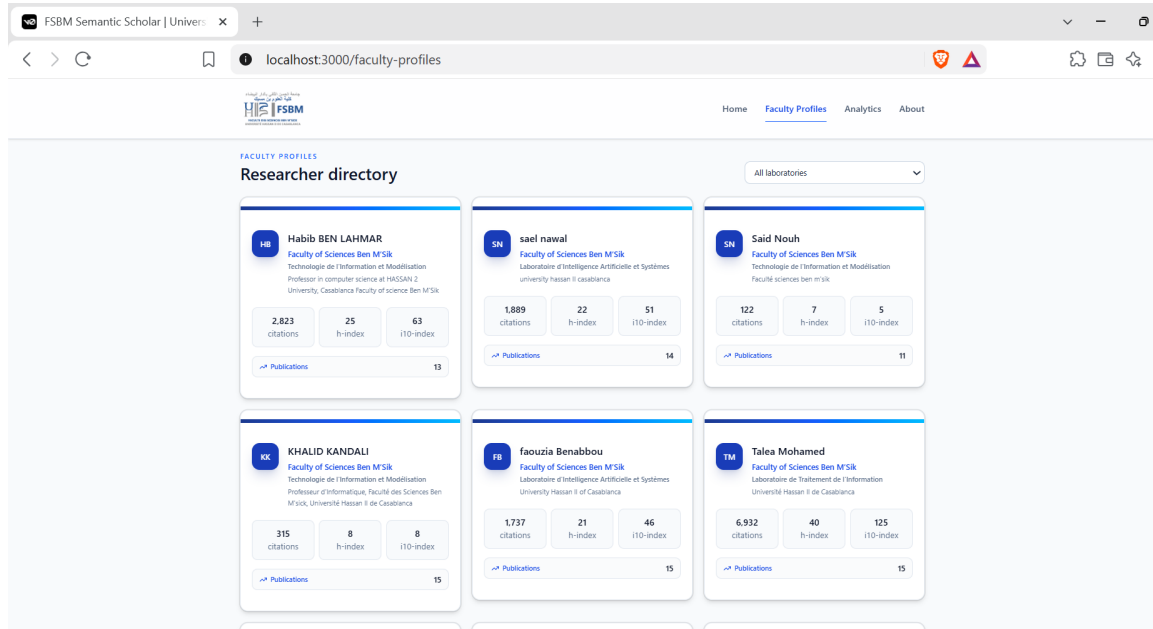


Figure 3.2: Faculty Researcher Directory displaying researcher affiliations, laboratory filters, and computed academic impact metrics (h -index, $i10$ -index, total citations).

As highlighted in Figure 3.2, each faculty member is presented as a modular summary card featuring:

- **Institutional Affiliation & Laboratory:** Affiliation badges linked to laboratory groupings (e.g., *Laboratoire d'Intelligence Artificielle et Systèmes*).
- **Bibliometric Scorecards:** Standardized display of key academic indicators including total career citations, h -index, and $i10$ -index.
- **Indexed Publication Count:** Exact number of cleansed, indexed publications available for downstream semantic matching.

3.3.3 Individual Researcher Profile

Clicking into a specific faculty profile navigates to an exhaustive academic overview view.

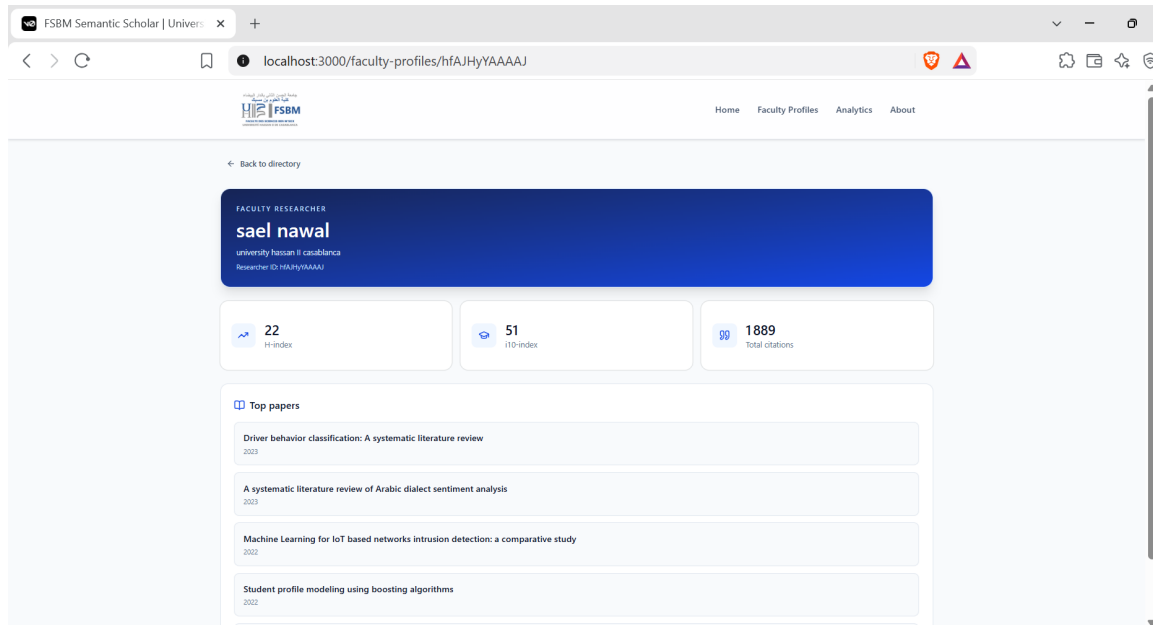


Figure 3.3: Individual researcher profile view displaying historical metrics and indexed top papers.

As depicted in Figure 3.3, the detailed view presents:

- **Header Banner:** Researcher name, institution, Google Scholar ID, and verified research laboratory.
- **Global Metrics Summary:** Responsive summary cards reflecting the researcher's overall academic footprint.
- **Top Papers List:** Chronologically ordered scientific publications, indexed abstracts, and direct reference links.

Chapter 4

Conclusion & Perspectives

4.1 Summary of Achievements

This project successfully demonstrates the integration of robust data engineering with modern AI search techniques. By automating the extraction of academic profiles, bypassing complex publisher paywalls using headless browsing, and indexing the data semantically via ChromaDB, we have created a highly scalable platform for exploring the scientific output of FSBM.

4.2 Future Work

Future iterations of this platform could include:

- **Automated Summarization:** Integrating Large Language Models (LLMs) to generate localized summaries of complex research papers.
- **Graph Visualization:** Implementing network graphs to visualize collaboration and citation ecosystems among university researchers.
- **Continuous Integration:** Deploying the scraping orchestrator as a scheduled CRON job on a cloud server to keep the database synchronized with new publications automatically.